

HYBRID AI SYSTEMS for the Modern AI PM

Classical ML + DL + LLMs + Agentic AI — Real-World Hybrid Scenarios
By Dr. Marily Nika | 2025 Market Edition

The 2025 AI market has moved beyond the 'LLM vs. ML' debate. The most powerful, cost-effective, and production-ready AI products are HYBRID — they combine classical ML models, deep learning pipelines, and LLM-based agents into a single orchestrated system. This guide teaches you, as an AI PM, exactly how to think about, design, and ship these systems.

Part 1 — The Hybrid AI Mental Model

Why Hybrid? The 2025 Market Reality

Pure LLM products are expensive, slow, and hallucination-prone for structured tasks. Pure ML products are rigid, can't reason, and can't handle open-ended inputs. The market has converged on a hybrid architecture where each component does what it does best.

Component	What It's Good At	What It's Bad At	Role in Hybrid System
Classical ML (XGBoost, RF, LR)	Fast, cheap, interpretable predictions on structured data	Unstructured input, reasoning, language	Specialist Tool / Signal Provider
Deep Learning (CNN, BERT, Whisper)	Unstructured data (images, audio, text embeddings)	Reasoning, instruction-following, cost	Perception Layer / Embedding Engine
LLM (GPT-4o, Claude, Gemini)	Reasoning, language, planning, synthesis	Structured prediction, speed, cost at scale	Orchestrator / Reasoner / Generator
Agentic AI (ReAct, Plan-Execute)	Multi-step tasks, tool use, dynamic workflows	Reliability, cost, latency, determinism	Workflow Coordinator
Rules / Heuristics	Deterministic, auditable, zero-latency decisions	Generalisation, edge cases	Guardrails / Fast Path

The Hybrid AI Architecture Principle

📌 **Dr. Marily's Core Principle:** Route each sub-task to the cheapest, fastest component that can solve it at the required quality level. LLMs are the brain. ML models are the specialist organs. Rules are the reflexes. The agent is the nervous system that connects them.

The Hybrid Decision Matrix — Which Layer Owns What?

Decision Type	Latency Need	Interpretability Need	Best Layer
Is this input safe / valid?	< 10ms	Low	Rules + ML Classifier
What type of request is this?	< 50ms	Medium	ML Intent Classifier
Extract structured data from text	< 200ms	Low	NER / DL Extraction Model
Score / rank / predict a number	< 100ms	High (regulated)	Classical ML Model
Reason about ambiguous context	< 3s	Low	LLM
Generate a response / plan	< 5s	Low	LLM
Execute multi-step workflow	< 30s	Medium	Agentic AI + Tools
Verify / validate LLM output	< 500ms	Medium	ML Classifier + Rules

Part 2 — The 8 Real-World Hybrid Scenarios

These are the exact patterns you will encounter in 2025 AI PM roles. Each scenario includes: the architecture, the PM decisions, the metrics, and the failure modes.

SCENARIO 01 ML Model as a Tool Inside an LLM Agent

The most common hybrid pattern in production today

What It Is

An LLM agent orchestrates a workflow. When it needs a precise, fast, or structured prediction — it calls a classical ML or DL model as a tool, exactly like it would call a search API or database.

Real-World Examples

Product	How ML + LLM Combine
Salesforce Einstein Copilot	LLM agent calls a churn prediction ML model as a tool to surface at-risk accounts during a sales conversation
GitHub Copilot	LLM calls a code vulnerability classifier (ML) before suggesting code completions
Healthcare AI (Nuance DAX)	LLM agent calls a medical NER model to extract diagnoses, then generates clinical notes
Fraud Investigation Agent	LLM agent calls a fraud scoring ML model, then reasons about the score to decide next action
E-commerce Personalisation	LLM agent calls a recommendation ranking model, then generates personalised copy around the ranked items

The Architecture

- Step 1: User sends a natural language request to the LLM agent
- Step 2: Agent parses intent and identifies that a structured prediction is needed
- Step 3: Agent calls the ML model tool with structured parameters (e.g. {customer_id: 123, feature_set: [...]})
- Step 4: ML model returns a score/prediction (e.g. churn_probability: 0.87, top_features: [...])
- Step 5: Agent incorporates the ML output into its reasoning context
- Step 6: Agent generates a natural language response grounded in the ML prediction

PM Decisions for This Pattern

Decision	Options	How to Decide
When does the agent call the ML tool?	Always / On-demand / Threshold-triggered	Always if prediction is always relevant; on-demand if agent should judge
How is ML output passed to LLM?	Raw score / Interpreted label / Full feature explanation	Regulated domains: full explanation. Consumer: interpreted label
What if ML model is unavailable?	Fail gracefully / LLM estimates / Block action	Define fallback in PRD — never let agent hallucinate a prediction
Who owns the ML model quality?	ML team / PM / Shared	PM owns the tool schema and quality bar; ML team owns the model

Key Metrics

- Tool Call Accuracy: % of times agent calls the ML tool with correct parameters
- ML Tool Latency: p95 latency of ML model inference (must fit within agent step budget)
- Grounding Rate: % of agent responses that correctly cite the ML prediction
- Hallucination-of-Prediction Rate: % of responses where agent invents a score instead of calling the tool

❗ **Failure Mode:** The LLM may hallucinate a prediction instead of calling the ML tool — especially if the tool description is vague. Fix: make the tool description explicit about WHEN to call it, and add output validation to detect invented scores.

SCENARIO 02 ML Router + LLM Specialist

Use ML to decide WHICH LLM (or no LLM) to call — the cost-efficiency pattern

What It Is

A lightweight ML classifier acts as a router. It analyses the incoming request and routes it to: a rules engine (for simple/known queries), a small/cheap LLM (for medium complexity), or a frontier LLM (for complex reasoning). This can cut LLM costs by 60–80%.

Real-World Examples

Company / Product	Routing Logic
Intercom Fin	ML classifier routes: FAQ → rules, billing → small LLM, complex complaint → GPT-4o
Notion AI	Complexity classifier: short formatting → fast model, long-form writing → frontier model
Klarna AI	Intent classifier: known transaction query → deterministic lookup, dispute → LLM agent
RouteLLM (open source)	Trained router model that predicts whether a query needs a strong or weak LLM

The Architecture

- Step 1: Request arrives → ML Router classifies complexity/intent (< 20ms)
- Step 2: Router outputs: ROUTE_RULES | ROUTE_SMALL_LLM | ROUTE_FRONTIER_LLM | ROUTE_AGENT
- Step 3: Request dispatched to appropriate handler
- Step 4: Response returned — user sees no difference in interface

PM Decisions

Decision	Consideration
What are the routing categories?	Define based on cost/quality analysis — start with 2 tiers, expand as data grows
What is the routing accuracy threshold?	Misrouting to cheap model = quality drop. Set minimum routing precision > 95% for each category
How do you handle routing errors?	Build a feedback loop: if user re-asks or overrides, log as routing error and retrain router
How do you measure routing ROI?	Cost per query before vs. after routing. Quality score per tier. % queries routed to each tier

 **Business Impact:** A well-tuned ML router can reduce LLM API costs by 60–80% with < 2% quality degradation. This is one of the highest-ROI PM decisions in AI product development.

SCENARIO 03 DL Perception Layer + LLM Reasoning Layer

Multimodal products: see with DL, think with LLM

What It Is

A deep learning model handles perception (understanding images, audio, video, documents). The DL model converts unstructured input into structured representations. The LLM then reasons over those representations to generate actions or responses.

Real-World Examples

Product	DL Layer	LLM Layer
GPT-4o Vision	Vision encoder (ViT) processes image → embeddings	LLM reasons over visual embeddings to answer questions
Waymo / Tesla FSD	CNN detects objects, lanes, signs → structured scene graph	Planning model reasons over scene graph for driving decisions
Medical AI (Rad AI)	CNN analyses X-ray/MRI → findings list	LLM generates radiology report from findings
Document AI (AWS Textract + Bedrock)	OCR + layout model extracts text/tables from PDF	LLM answers questions about extracted content
Customer Service with Screen Share	CV model reads customer's screen → structured state	LLM agent guides customer based on screen state
Retail Visual Search	CNN encodes product image → embedding	LLM generates product description + recommendation copy

PM Decisions

Decision	Options	PM Guidance
What does DL output to LLM?	Raw embeddings / Structured JSON / Natural language description	Structured JSON is most reliable; natural language adds latency but improves LLM reasoning
Where is the quality bar set?	DL accuracy vs. LLM reasoning quality	DL errors compound — a 90% accurate CV model + 90% accurate LLM = 81% end-to-end accuracy
Latency budget allocation	DL inference time + LLM inference time	DL should consume < 30% of total latency budget
Fallback when DL fails?	Skip DL, pass raw input to LLM / Block / Human review	Define in PRD — never silently degrade

Key Insight: The quality of the DL perception layer is the ceiling for the entire system. A brilliant LLM cannot reason correctly over bad DL outputs. As PM, invest in DL quality first — it has compounding returns.

SCENARIO 04 LLM Agent + ML Guardrails

Use ML to constrain, validate, and protect LLM outputs

What It Is

The LLM generates freely. ML classifiers act as guardrails — validating inputs before they reach the LLM, and validating outputs before they reach the user. This is the safety architecture used by every serious production AI product.

The 3-Layer Guardrail Architecture

Layer	ML Model Type	What It Catches	Action on Trigger
Input Guardrail	Multi-label classifier	Prompt injection, jailbreaks, PII, harmful intent, off-topic	Block / Redact / Reroute
LLM Core	LLM (GPT-4o, Claude, etc.)	Generates response	—
Output Guardrail	Multi-label classifier + NER	Hallucinations, PII leakage, harmful content, policy violations, factual errors	Block / Redact / Flag for review

Real-World Examples

Product	Guardrail ML Models Used
OpenAI Moderation API	Fine-tuned classifier for 11 harm categories — used as input/output guardrail
Anthropic Claude (Constitutional AI)	Reward model (ML) trained on human preferences acts as output quality guardrail
Microsoft Azure AI Content Safety	Ensemble of ML classifiers for hate, violence, sexual, self-harm content
Salesforce Einstein Trust Layer	PII detection model (NER) + toxicity classifier wrap every LLM call
Llama Guard (Meta)	Fine-tuned Llama model acting as a safety classifier for both input and output

PM Decisions

Decision	Consideration
Build vs. buy guardrails?	Buy (OpenAI Moderation, Azure Content Safety) for standard cases. Build custom for domain-specific risks (medical, legal, finance)
Precision vs. Recall for guardrails?	Input guardrails: high recall (catch everything). Output guardrails: high precision (avoid false blocks that frustrate users)
Synchronous vs. async guardrails?	Sync: adds latency but blocks bad outputs. Async: faster UX but bad outputs may reach user briefly
What to tell the user when blocked?	Always explain why (vaguely) and offer an alternative path. Silent blocks destroy trust

SCENARIO 05 Multi-Agent System with Specialist ML Agents

Orchestrator LLM delegates to specialist ML/DL sub-agents

What It Is

A manager LLM agent breaks down a complex task and delegates sub-tasks to specialist agents. Some specialist agents are LLM-based. Others are classical ML or DL models wrapped as agents. This is the architecture behind the most powerful AI products of 2025.

The Architecture Pattern

- ORCHESTRATOR: LLM agent — receives user goal, creates plan, delegates sub-tasks, synthesises results
- SPECIALIST AGENT A: LLM-based — handles language, reasoning, generation sub-tasks
- SPECIALIST AGENT B: ML model wrapped as agent — handles prediction, scoring, classification sub-tasks
- SPECIALIST AGENT C: DL model wrapped as agent — handles perception, embedding, extraction sub-tasks
- SPECIALIST AGENT D: Rules/API — handles deterministic lookups, database queries, calculations

Real-World Examples

Product	Orchestrator	Specialist Agents
Salesforce Agentforce	LLM orchestrator	CRM ML model (churn/upsell), NER agent (entity extraction), Rules agent (pricing lookup)
Harvey AI (Legal)	LLM orchestrator	Contract NER agent, clause classification ML agent, precedent search DL agent, LLM drafting agent
Agentic RAG (Enterprise Search)	LLM orchestrator	Query rewriting LLM, dense retrieval DL agent, reranking ML agent, answer generation LLM
Clinical Decision Support	LLM orchestrator	Medical NER agent, diagnosis classification ML agent, drug interaction rules agent, report generation LLM
Financial Analysis Agent	LLM orchestrator	Sentiment ML agent (news), anomaly detection ML agent, forecasting DL agent, narrative LLM agent

PM Decisions for Multi-Agent Hybrid Systems

Decision	Options	Guidance
Agent decomposition	How to split the task into sub-agents?	Split by modality (text/image/structured), by latency tier (fast/slow), or by risk level (safe/human-reviewed)
Orchestrator model choice	Frontier LLM vs. mid-tier LLM	Orchestrator needs strong reasoning — use frontier. Specialists can be cheaper
State management	Shared memory vs. message passing	Define explicit state schema in PRD — agents must not infer state from conversation history alone
Error propagation	Fail fast vs. graceful degradation	Define: if specialist agent fails, can orchestrator proceed with partial results or must it halt?
Cost attribution	How to track cost per agent?	Instrument each agent call separately — you need to know which agent is the cost driver

🔗 **Dr. Marily's Rule:** In a multi-agent system, the orchestrator is only as good as its specialist agents. Define the contract (input schema, output schema, latency SLA, quality bar) for EACH specialist agent in your PRD — treat each one as a mini-product.

SCENARIO 06 LLM for Cold Start, ML for Scale

Use LLM to bootstrap, then distil into a cheaper ML model

What It Is

LLMs are expensive at scale but excellent at zero-shot tasks. The pattern: use an LLM to label, classify, or generate outputs for a large dataset, then train a smaller, faster, cheaper ML/DL model on those LLM-generated labels. Ship the small model at scale.

The 4-Phase Process

Phase 1: LLM Labelling: Use GPT-4o or Claude to label/classify/score a large unlabelled dataset. Cost: high. Speed: slow. Quality: high.

Phase 2: Quality Audit: Human review of 5–10% of LLM labels. Measure label accuracy. If > 90%, proceed. If not, refine prompt.

Phase 3: Train Small Model: Train a BERT/DistilBERT/XGBoost model on LLM-generated labels. This is 'LLM distillation'.

Phase 4: Deploy Small Model: Ship the small model at scale. 100x cheaper, 10x faster, same quality. LLM only used for edge cases.

Real-World Examples

Use Case	LLM Role	Distilled Model
Content moderation at scale	GPT-4o labels 1M posts for 20 harm categories	Fine-tuned DistilBERT — 50ms, \$0.00001/query
Customer intent classification	Claude labels 500K support tickets with intent + sentiment	XGBoost classifier — real-time routing
Product attribute extraction	LLM extracts attributes from 10M product descriptions	NER model — batch processing pipeline
Code quality scoring	GPT-4o scores 100K code snippets on 5 quality dimensions	Fine-tuned CodeBERT — CI/CD integration
Medical entity normalisation	LLM maps 1M clinical terms to standard ontology (SNOMED)	Embedding similarity model — real-time EHR processing

📌 **Cost Impact:** This pattern can reduce per-query AI cost by 95–99% while maintaining 90–95% of LLM quality. It is the most important cost optimisation strategy for AI products at scale.

SCENARIO 07 Retrieval-Augmented Generation with ML Reranking

Hybrid RAG: DL retrieval + ML reranking + LLM generation

What It Is

Standard RAG uses a single embedding model for retrieval. Hybrid RAG adds a reranking ML model between retrieval and generation — dramatically improving answer quality. This is the architecture used by enterprise search products in 2025.

The Hybrid RAG Stack

Stage	Component	Model Type	PM Quality Lever
Query Understanding	Query rewriting / expansion	LLM (small)	Improves recall for ambiguous queries
Sparse Retrieval	BM25 / keyword search	Rules-based	Fast, interpretable, good for exact matches
Dense Retrieval	Embedding similarity search	DL (E5, BGE, Cohere Embed)	Good for semantic similarity
Hybrid Fusion	Combine sparse + dense results	RRF (Reciprocal Rank Fusion)	Best of both retrieval methods
Reranking	Re-score top-K results for relevance	ML Cross-Encoder (Cohere Rerank, BGE Reranker)	Biggest quality gain — +15–25% relevance
Generation	Answer generation grounded in context	LLM (GPT-4o, Claude)	Faithfulness + completeness
Output Validation	Verify answer is grounded in retrieved context	ML Faithfulness Classifier	Hallucination prevention

PM Decisions

Decision	Guidance
Sparse vs. Dense vs. Hybrid retrieval?	Always start with hybrid — sparse catches exact matches, dense catches semantic matches. Pure dense misses keyword queries.
Do you need a reranker?	Yes, if your knowledge base > 10K documents or query diversity is high. Reranker adds 100–300ms but +15–25% quality.
How many chunks to retrieve (top-K)?	Retrieve top-20, rerank to top-5. More retrieval = better recall. Reranking = better precision.
Faithfulness validation — sync or async?	Sync for high-stakes domains (medical, legal, finance). Async for consumer products.

SCENARIO 08 Real-Time ML Signals + LLM Personalisation

Combine real-time ML predictions with LLM-generated personalised experiences

What It Is

ML models compute real-time user signals (propensity scores, affinity scores, risk scores). These signals are injected into the LLM's context to personalise responses, recommendations, and actions at the individual user level — at scale.

Real-World Examples

Product	ML Signal	LLM Personalisation
Duolingo Max	ML model: user's weak grammar areas, learning velocity	LLM generates personalised explanations targeting exact weaknesses
Spotify DJ (AI)	ML model: mood prediction, listening context, time-of-day affinity	LLM generates personalised DJ commentary and transitions
HubSpot AI	ML model: lead score, deal stage probability, churn risk	LLM generates personalised outreach copy calibrated to lead score
Khan Academy Khanmigo	ML model: knowledge graph mastery per concept	LLM Socratic tutor adapts question difficulty to mastery level
Retail AI (Stitch Fix)	ML model: style affinity, size prediction, price sensitivity	LLM generates personalised style notes for each item in the box

The Architecture

- Step 1: User action triggers real-time ML feature computation (< 50ms)
- Step 2: ML model outputs user signals as structured JSON (scores, labels, top features)
- Step 3: Signals injected into LLM system prompt as personalisation context
- Step 4: LLM generates response personalised to the user's ML-derived profile
- Step 5: User interaction logged → updates ML model features (data flywheel)

PM Decisions

Decision	Guidance
Which ML signals to inject?	Only inject signals that meaningfully change the LLM response. More signals = more tokens = more cost. Test each signal's marginal quality lift.
How to represent ML signals to LLM?	Natural language > raw numbers. 'This user is highly price-sensitive' > 'price_sensitivity_score: 0.87'
Privacy: can ML signals be in LLM context?	Check if ML signals contain PII or sensitive inferences. Define data minimisation rules in PRD.
How to measure personalisation quality?	A/B test: personalised LLM vs. generic LLM. Measure task completion, engagement, conversion delta.

Part 3 — The Hybrid AI PM Framework

The COMPOSE Framework — Dr. Marily's Hybrid Design Method

Use this framework every time you are designing a hybrid AI system. It ensures you make deliberate decisions about each component rather than defaulting to 'just use an LLM'.

C — Classify the Sub-Tasks

Break the user's goal into atomic sub-tasks. For each sub-task, classify it: Is it a prediction? A perception task? A reasoning task? A generation task? A lookup?

O — Optimise Component Selection

For each sub-task, select the cheapest/fastest component that meets the quality bar. Rules → ML → DL → Small LLM → Frontier LLM → Agent. Never over-engineer.

M — Map the Data Contracts

Define the input/output schema for every component interface. What does Component A pass to Component B? What format? What validation?

P — Plan for Failure at Every Seam

Every interface between components is a failure point. Define: what happens if Component B is unavailable? Slow? Returns unexpected output?

O — Observe with Layered Metrics

Instrument every component separately. You need to know which layer is the quality/cost/latency bottleneck. Don't just measure end-to-end — measure each seam.

S — Set the Human Oversight Points

Identify which decisions in the pipeline are irreversible or high-stakes. Insert human review gates at those points. Define the escalation path.

E — Evaluate End-to-End, Not Just Components

Each component may pass its own quality bar but the system can still fail. Build an end-to-end Golden Dataset that tests the full pipeline, not just individual components.

The Hybrid AI PRD — Additional Sections

Your existing AI PRD template covers LLM-specific sections. Add these sections for every hybrid system:

📄 **Component Architecture Diagram:** A table or diagram showing every component, its type (Rules/ML/DL/LLM/Agent), its input/output schema, its latency budget, and its quality bar.

📄 **Inter-Component Data Contracts:** For each interface between components: input schema, output schema, error handling behaviour, and fallback strategy.

📄 **Latency Budget Allocation:** Total end-to-end latency SLO broken down by component. E.g. Input guardrail: 20ms | ML router: 15ms | LLM: 2000ms | Output guardrail: 30ms | Total: 2065ms

📄 **Cost Model by Component:** Monthly cost broken down by component. ML inference cost + LLM token cost + DL inference cost + orchestration overhead.

📄 **Failure Mode Matrix:** For each component: what are the failure modes? What is the fallback? What does the user see? Who is alerted?

📄 **Quality Compounding Analysis:** Calculate end-to-end accuracy as the product of component accuracies. E.g. $95\% \times 90\% \times 88\% = 75.2\%$ end-to-end. This is your quality floor.

Hybrid AI Metrics Dashboard — What to Track

Metric	Layer	What It Catches	Alert Threshold
Component Latency (p95)	Each component	Bottleneck identification	Exceeds budget allocation
Tool Call Success Rate	Agent → ML/DL tools	ML tool availability/reliability	< 99.5%
Routing Accuracy	ML Router	Misrouted queries (quality/cost impact)	< 95% precision per route
Grounding Rate	LLM + ML signal	LLM ignoring ML tool outputs	< 90%
Quality Compounding Score	End-to-end	Cascading quality degradation	< 80% of baseline
Cost per Component	Each component	Cost driver identification	Exceeds budget allocation
Fallback Trigger Rate	Each interface	Component reliability issues	> 1% of requests
End-to-End Task Completion	Full pipeline	System-level quality	< target from PRD

The Hybrid AI Failure Mode Taxonomy

Failure Mode	Where It Happens	Symptom	Prevention
Quality Compounding	Multi-component pipeline	End-to-end quality much worse than any single component	Measure end-to-end, not just components
Latency Stacking	Sequential components	Total latency exceeds SLO even though each component is within budget	Define and enforce per-component latency budgets
Context Poisoning	ML signal → LLM context	Bad ML prediction corrupts LLM reasoning for entire session	Validate ML outputs before injecting into LLM context
Tool Hallucination	LLM agent → ML tool	LLM invents ML prediction instead of calling tool	Strict tool schemas + output validation
Routing Cascade Failure	ML router → downstream	Router misclassifies → wrong model → bad output → user complaint	Monitor routing accuracy + downstream quality per route
Fallback Invisibility	Any component	Component silently fails, fallback activates, quality drops undetected	Alert on every fallback trigger, not just hard failures
Data Contract Drift	Component interfaces	Upstream component changes output schema, downstream breaks silently	Schema validation at every interface + contract tests

Part 4 — Quick Reference

Hybrid AI Scenario Selector

If your product needs...	Use this Hybrid Pattern
Precise structured predictions inside a conversational AI	Scenario 1: ML as Tool inside LLM Agent
Cost reduction on high-volume LLM queries	Scenario 2: ML Router + LLM Specialist
Understanding images, audio, or documents + reasoning	Scenario 3: DL Perception + LLM Reasoning
Safe, policy-compliant LLM outputs at scale	Scenario 4: LLM Agent + ML Guardrails
Complex multi-step workflows with diverse sub-tasks	Scenario 5: Multi-Agent with Specialist ML Agents
Scale a task that LLM does well but is too expensive at volume	Scenario 6: LLM Cold Start → ML Distillation
High-quality enterprise knowledge retrieval	Scenario 7: Hybrid RAG with ML Reranking
Personalised AI experiences at individual user level	Scenario 8: Real-Time ML Signals + LLM Personalisation

The 10 Hybrid AI PM Interview Questions (2025)

Q: How would you decide when to use an ML model vs. an LLM for a sub-task?

→ Route to cheapest/fastest component that meets quality bar. ML for structured prediction, LLM for reasoning/generation. Always measure cost of being wrong.

Q: How would you design a multi-agent system for [use case]?

→ Apply COMPOSE: decompose tasks, select components, define data contracts, plan failure modes, set oversight points, evaluate end-to-end.

Q: How do you prevent an LLM agent from hallucinating ML predictions?

→ Strict tool schemas, output validation layer, grounding rate metric, fallback to explicit error message if tool unavailable.

Q: How would you reduce LLM costs by 70% without sacrificing quality?

→ ML router pattern: classify query complexity, route simple queries to rules/small model, complex to frontier LLM. Measure quality per route.

Q: How do you measure quality in a hybrid system with 5 components?

→ Measure each component separately AND end-to-end. Quality compounds multiplicatively. Build end-to-end Golden Dataset.

Q: What is the biggest risk in a multi-agent hybrid system?

→ Quality compounding + context poisoning + tool hallucination. Mitigate with data contracts, output validation, and end-to-end evals.

Q: How would you build a RAG system that doesn't hallucinate?

→ Hybrid retrieval (sparse + dense) + ML reranker + faithfulness classifier on output + source citations in UX.

Q: How do you personalise an LLM product at scale without it being too expensive?

→ Real-time ML signals injected as context. ML computes personalisation cheaply; LLM uses it. Data flywheel closes the loop.

Q: How do you handle a component failure in a hybrid pipeline?

→ Define fallback for every component in PRD. Alert on fallback triggers. Never silently degrade. User always gets a response.

Q: How do you know which component is the bottleneck in a hybrid system?

→ Instrument each component separately. Track latency, cost, quality, and error rate per component. The bottleneck is the one with the worst metric relative to its budget.